

EL ECOSISTEMA EN LA NUBE

K.V.Ramirez Ambrocio

7690-22-2239 Universidad Mariano Gálvez

Seminario de Tecnologías de Información

kramireza10@miumg.edu.gt

Resumen

El presente ensayo tiene como objetivo explicar cómo se relacionan cinco elementos clave en el despliegue de aplicaciones modernas en la nube: la orquestación de servidores, Kubernetes, los microservicios, el protocolo OAuth 2.0 y la metodología de aplicaciones de doce factores (12-factor app). Como resultado se identificó que estos cinco conceptos no son piezas aisladas, sino que trabajan en conjunto dentro de una misma arquitectura: los microservicios dividen la aplicación en partes pequeñas, Kubernetes se encarga de orquestar y mantener con vida esas partes automáticamente, OAuth 2.0 protege el acceso entre esos servicios y con los usuarios, y la metodología de doce factores da las reglas de diseño para que toda la aplicación se comporte bien dentro de un entorno en la nube. Se concluye que entender estos conceptos por separado no es suficiente; lo que realmente permite construir una aplicación moderna, escalable y seguro es comprender cómo encajan entre sí.

Palabras clave: orquestación de contenedores, Kubernetes, microservicios, OAuth 2.0, aplicaciones en la nube.

Desarrollo del tema

Cuando una aplicación crece y empieza a recibir mucho tráfico de datos, ya no es suficiente con tenerla corriendo en un solo servidor. Ahí es donde entra la orquestación de servidores, que es básicamente el proceso de automatizar el despliegue, la administración y el escalamiento de muchas piezas de software que trabajan juntas. En lugar de que una persona tenga que revisar manualmente si un servicio se cayó o si hace falta levantar más instancias por el aumento de usuarios, la orquestación se encarga de hacerlo sola, siguiendo reglas definidas previamente (Red Hat, 2026).

La herramienta más usada hoy en día para lograr esto es Kubernetes, también conocida como K8s. Kubernetes es una plataforma de código abierto que automatiza el despliegue, el escalamiento y la administración de aplicaciones que corren dentro de contenedores (Kubernetes Authors, s.f.). Un contenedor es, en pocas palabras, una forma de empaquetar una aplicación junto con todo lo que necesita para funcionar, para que corra igual sin importar en qué computadora o servidor se ejecute. Lo interesante de Kubernetes es que funciona de forma declarativa: en lugar de decirle paso a paso qué hacer, la persona desarrolladora describe cómo quiere que se vea el sistema al final, y Kubernetes se encarga de comparar constantemente ese estado deseado contra el estado real, corrigiendo automáticamente cualquier diferencia (Mirantis, 2026). Por ejemplo, si un contenedor se cae, Kubernetes lo reinicia solo; si un servidor falla, mueve las cargas de trabajo a otro que sí esté disponible; y si el tráfico aumenta, puede crear más copias del servicio automáticamente.

Todo esto conecta directamente con los microservicios, un estilo de arquitectura donde en lugar de tener una sola aplicación gigante, esta se divide en varios servicios pequeños e independientes,

cada uno responsable de una función específica del negocio. La orquestación con Kubernetes es precisamente lo que hace viable trabajar con microservicios a gran escala, porque coordina esos servicios pequeños dentro de contenedores separados, permitiendo que cada uno escale según su propia demanda sin afectar a los demás (Red Hat, 2026). Sin una herramienta de orquestación, manejar decenas o cientos de microservicios manualmente sería prácticamente imposible.

Ahora bien, tener muchos servicios pequeños comunicándose entre sí trae un problema importante: ¿cómo se asegura que solo las personas y aplicaciones autorizadas puedan acceder a cada uno de esos servicios? Aquí es donde entra el protocolo OAuth 2.0 que permite que una aplicación de terceros obtenga acceso limitado a un servicio esto es justo lo que pasa cuando alguien inicia sesión en una aplicación usando su cuenta de Google: la aplicación nunca ve la contraseña de Google, solo recibe un token con permisos limitados para lo que esa aplicación realmente necesita hacer. En arquitecturas de microservicios, OAuth 2.0 también se usa para que los propios servicios se autenticuen entre ellos antes de compartir información.

Por último la metodología de doce factores, conocida como 12-factor app. Fue creada por desarrolladores de Heroku como un conjunto de doce principios para construir aplicaciones que funcionen bien como servicio en la nube, que sean fáciles de escalar y que minimicen las diferencias entre el entorno de desarrollo y el de producción (12factor.net, s.f.). Algunos de estos principios son especialmente relevantes cuando se trabaja con Kubernetes y microservicios: por ejemplo, el principio de mantener la configuración fuera del código, el de tratar los procesos como algo sin estado que se puede reiniciar en cualquier momento, y el de exponer los servicios a través de un puerto, sin depender de un servidor web específico inyectado en tiempo de ejecución. Estos principios encajan perfectamente con la forma en que Kubernetes maneja los contenedores, porque un contenedor que sigue estas reglas es mucho más fácil de reiniciar, mover o escalar automáticamente.

Al final, lo que se ve es que estos cinco conceptos no compiten entre sí, sino que se complementan dentro de una misma arquitectura moderna: se construye la aplicación en microservicios pequeños siguiendo los principios de doce factores, se empaquetan en contenedores, se orquestan con Kubernetes para que se mantengan disponibles y escalen automáticamente, y se protege el acceso entre todos esos servicios y con los usuarios finales usando OAuth 2.0.

Observaciones y comentarios

Algo que llama la atención al estudiar estos temas juntos es que ninguno de ellos resuelve un problema por completo si se usa de forma aislada. Por ejemplo, tener microservicios sin una herramienta de orquestación como Kubernetes puede volverse un caos de administración manual, y tener Kubernetes sin seguir principios como los de doce factores puede generar contenedores difíciles de escalar porque guardan estado o dependen de configuraciones específicas del servidor donde corren.

También se observa que OAuth 2.0, a pesar de ser un estándar de 2012, sigue siendo la base de la seguridad en casi cualquier aplicación moderna que permita “iniciar sesión con Google.” “iniciar sesión con email”, lo cual demuestra que un buen estándar técnico puede mantenerse vigente por más de una década, aunque la tecnología que lo rodea cambie constantemente.

Conclusiones

1. La orquestación de servidores automatiza tareas que antes se hacían manualmente, como reiniciar servicios caídos o escalar según la demanda, y es indispensable cuando una aplicación crece más allá de un solo servidor.
2. Kubernetes funciona de forma declarativa: la persona desarrolladora define el estado deseado del sistema, y la plataforma se encarga de mantenerlo así de manera automática y constante.
3. Los microservicios y Kubernetes son complementarios: dividir una aplicación en servicios pequeños solo es manejable a gran escala si existe una herramienta de orquestación que los coordine.
4. OAuth 2.0 resuelve el problema de compartir acceso sin compartir contraseñas, usando tokens temporales con permisos limitados, y es la base de la seguridad en la comunicación entre servicios y usuarios.
5. La metodología de doce factores aporta las reglas de diseño que hacen que una aplicación funcione bien dentro de contenedores orquestados, por lo que no debe verse como un tema aparte, sino como la base que conecta a todos los demás conceptos.

Referencias

- 12factor.net. (s.f.). *The twelve-factor app*. <https://12factor.net/>
- Hardt, D. (Ed.). (2012). *The OAuth 2.0 authorization framework* (RFC 6749). Internet Engineering Task Force. <https://www.rfc-editor.org/info/rfc6749/>
- Kubernetes Authors. (s.f.). *Production-grade container orchestration*. Cloud Native Computing Foundation. <https://kubernetes.io/docs/home/>
- Mirantis. (2026). *What is Kubernetes orchestration?* <https://www.mirantis.com/cloud-native-concepts/getting-started-with-kubernetes/what-is-kubernetes-orchestration/>
- Red Hat. (2026, 10 de julio). *What is container orchestration?* <https://www.redhat.com/en/topics/containers/what-is-container-orchestration>
- Repositorio Git: <https://github.com/Valkill/seminario-encuesta.git>